

# Bases de Datos a Gran Escala

## Sesión 2 · SQL (ii): DDL

Diego Sevilla Ruiz

Departamento de Ingeniería y Tecnología de Computadores

Facultad de Informática · Universidad de Murcia

dsevilla@um.es

# SQL (ii): DDL

# Introducción

---

- SQL se divide lógicamente en dos sublenguajes:
  - **DML** (*Data Manipulation Language*), visto en la sesión anterior
  - **DDL** (*Data Definition Language*), el sublenguaje de definición del esquema de los datos
- En esta sesión veremos también la parte de actualización, modificación y borrado de datos de DML
- Veremos cómo usar JSON en SQL
- Finalmente, una guía de diseño de esquemas orientados a la eficiencia

# Data Definition Language (DDL)

---

- Permite describir los metadatos (tablas y relaciones entre ellas) de la base de datos
- Tablas, dominios de los atributos y restricciones de integridad
- Al conjunto de especificaciones se le conoce como **esquema** de la base de datos
- Por lo tanto, define también la sintaxis de las operaciones de inserción ( `INSERT` ) y actualización ( `UPDATE` )

# CREATE TABLE

- La construcción principal de SQL para la creación del esquema es la sentencia `CREATE TABLE`
- Para cada tabla, especifica sus columnas, dominios de columna y restricciones

```
CREATE TABLE nombre (  
    att1 tipo1 [[NOT] NULL] [UNIQUE],  
    att2 tipo2,  
    ...  
    PRIMARY KEY(...),  
    FOREIGN KEY attx REFERENCES Tabla(atty)  
        ON DELETE opción  
        ON UPDATE opción,  
    ...  
);
```

# CREATE TABLE: columnas

---

- Cada campo lista su tipo (se ven a continuación)
- Opcionalmente se puede especificar si la columna admite valores nulos ( `NULL` ) o no ( `NOT NULL` )
- También se puede indicar que la columna contiene valores que no se repetirán, con `UNIQUE`
- Se le puede asignar un valor por defecto al insertar datos, con `DEFAULT x`
- No se muestra en el ejemplo, pero a un atributo también se le puede añadir `PRIMARY KEY`

# CREATE TABLE: claves ajenas

---

- Opcionalmente, se pueden añadir restricciones de integridad con `FOREIGN KEY`
  - Lista un conjunto de atributos de la tabla (**clave ajena**)
  - Que hace referencia a otro conjunto de atributos de otra tabla (**clave**)
  - Opcionalmente se puede especificar qué sucede cuando el elemento referenciado se elimina ( `ON DELETE` ) o se le cambia el valor ( `ON UPDATE` )
  - Se estudian las opciones con detalle a continuación

# Creación de tablas a partir de consultas

---

- La construcción `CREATE TABLE` también permite crear tablas a partir de consultas
- La sintaxis es `CREATE TABLE t AS SELECT ... :`

```
CREATE TABLE T AS SELECT Id, Title FROM Posts;
```

# Tipos de datos: caracteres y enteros

---

- Caracteres:
  - `char(n)` un conjunto de `n` caracteres exactamente
  - `varchar(n)` un conjunto variable de como máximo `n` caracteres
  - `text` un conjunto de caracteres sin límite; no es parte del estándar SQL, pero lo soportan casi todos los SGBDR
- Números enteros: `integer`, `smallint`, `bigint`

| Tipo                  | Tamaño  | Rango                                                   |
|-----------------------|---------|---------------------------------------------------------|
| <code>smallint</code> | 2 bytes | -32 768 a +32 767                                       |
| <code>integer</code>  | 4 bytes | -2 147 483 648 a +2 147 483 647                         |
| <code>bigint</code>   | 8 bytes | -9 223 372 036 854 775 808 a +9 223 372 036 854 775 807 |

# Tipos de datos: números, mapas de bits y fechas

---

- Números de coma flotante:
  - `REAL(m, d)` (4 bytes), `FLOAT(m, d)` (4 bytes), `DOUBLE PRECISION(m, d)` (8 bytes), donde `m` es el número total de dígitos y `d` el número de dígitos decimales
  - La especificación de dígitos es opcional
- Mapas de bits: `BIT(n)`, `BIT VARYING(n)`
- Fechas: `DATE`, `TIME`, `DATETIME`, `TIMESTAMP`, `INTERVAL`, `YEAR` — se ven en detalle a continuación

# Fechas

- La gestión de fechas cambia de un SGBDR a otro; la vemos para MySQL
- Para ver los valores, podemos fijarnos en su «valor cero»:

| Tipo de dato | Valor cero            |
|--------------|-----------------------|
| DATE         | '0000-00-00'          |
| TIME         | '00:00:00'            |
| DATETIME     | '0000-00-00 00:00:00' |
| TIMESTAMP    | '0000-00-00 00:00:00' |

TIMESTAMP contiene una fecha compacta en el rango '1970-01-01 00:00:01' UTC a '2038-01-19 03:14:07' UTC

# Fechas: funciones

---

- MySQL ofrece funciones `YEAR()` , `MONTH()` , etc. para extraer datos particulares
- Funciones de búsqueda con intervalos (similares a ANSI-SQL):  
`INTERVAL` , `BETWEEN` , etc.
- `CURRENT_DATE()` , `CURRENT_TIME()` , `CURRENT_TIMESTAMP()` , etc. devuelven el tiempo actual con el tipo correspondiente

Posts desde hace una semana:

```
SELECT * FROM Posts WHERE CreationDate >= CURRENT_DATE() - INTERVAL 1 WEEK;
```

(se puede usar `WEEK` , `DAY` , `HOURL` , etc.)

# Fechas: rangos

---

```
SELECT * FROM Posts WHERE CreationDate  
    BETWEEN CURRENT_DATE() - INTERVAL 1 MONTH  
    AND CURRENT_DATE() - INTERVAL 1 WEEK;
```

Entre fechas específicas: las cuatro semanas anteriores, sin contar la última.

# El operador `CAST()`

- SQL permite cambiar el tipo de una expresión con `CAST()`
- Los tipos permitidos son los vistos, aunque hay cierta variación
- Por ejemplo, para un `INTEGER` siempre hay que decir si es `SIGNED` o `UNSIGNED`
- También se permite el tipo `CHAR`:

```
> SELECT CONCAT('ABC', CAST(1.1 AS CHAR(200))) as ABC;  
ABC1.1  
> SELECT CAST('1.1' as UNSIGNED) AS C;  
1.1  
> SELECT CAST('1.1' as DECIMAL(10,5)) AS C;  
1.10000
```

# Aseguramiento de la integridad

---

- Las partículas `FOREIGN KEY` de `CREATE TABLE` sirven para especificar las restricciones de integridad
- En cada tipo de modificación ( `UPDATE` y `DELETE` ) hay una posibilidad de trasladar cambios en cascada
- Estas modificaciones se refieren a la eliminación o actualización de **una fila que referencia a una fila eliminada o actualizada**

# Aseguramiento de la integridad: opciones

---

Cuatro posibilidades:

- `NO ACTION` (por defecto). No permite la modificación: el gestor lanza una excepción y no deja hacer el cambio, para mantener la consistencia/integridad
- `CASCADE` . Elimina (o actualiza) todas las filas que referencian a la fila modificada (las que tienen su clave ajena apuntando a esta fila)
- `SET DEFAULT`
- `SET NULL` . Establece el valor por defecto o nulo (la columna de la clave ajena debe permitirlo) al eliminar o modificar la fila

# Ejemplo de cascada

---

```
> CREATE TABLE Factura
  (Id INT PRIMARY KEY, Cliente TEXT, Total NUMBER);
> CREATE TABLE ItemFactura
  (Id INT , FactId INT, Nombre Text,
   Cantidad Int, Precio NUMBER,
   PRIMARY KEY(Id,FactId),
   FOREIGN KEY (FactId) REFERENCES Factura(Id) ON DELETE CASCADE);

> INSERT INTO Factura VALUES (5, "Pedro", 200.0);
> INSERT INTO ItemFactura VALUES (1, 5, "Tuerca", 2, 100);
```

# Ejemplo de cascada: efecto del DELETE

```
> SELECT * FROM Factura;
Id      Cliente      Total
-----
5       Pedro        200
> SELECT * FROM ItemFactura;
Id      FactId      Nomb  Cant  Prec
-----
1       5          Tuerca  2    100

> DELETE FROM Factura WHERE Id = 5;
> SELECT COUNT(*) FROM ItemFactura;
0
```

Al borrar la factura, **ON DELETE CASCADE** arrastra sus líneas.

# Alteración del esquema

---

- Una vez creada la tabla se puede alterar: añadir columnas, restricciones, etc.
- Esto tiene un impacto en la base de datos (consultas en ejecución, bloqueos, recálculos como en el caso de añadir o modificar índices, etc.)

Añadir columnas:

```
ALTER TABLE tabla ADD columna tipo [DEFAULT valor];
```

# Alteración del esquema: columnas

---

Eliminar columnas:

```
ALTER TABLE tabla DROP columna;
```

Modificar una columna:

```
ALTER TABLE tabla MODIFY columna tipo;
```

Cambiar el nombre y definición de una columna:

```
ALTER TABLE tabla CHANGE columna_anterior columna_nueva tipo;
```

# Alteración del esquema: restricciones

---

Añadir una clave primaria:

```
ALTER TABLE tabla ADD CONSTRAINT PRIMARY KEY columna;
```

Añadir una clave ajena:

```
ALTER TABLE tabla ADD FOREIGN KEY (columna) REFERENCES tabla_(columna_);
```

Eliminar una tabla:

```
DROP TABLE tabla;
```

# Alteración del esquema: columnas calculadas

---

También se pueden añadir campos calculados (también en `CREATE TABLE`):

```
ALTER TABLE tabla ADD columna <tipo>  
GENERATED ALWAYS AS (<expresión>) VIRTUAL | STORED;
```

- `VIRTUAL` indica que el valor se calcula al acceder a la columna (en general más lento)
- `STORED` indica que se calcula al insertar o actualizar la fila, por lo que ocupa espacio en disco
- Sólo las columnas `STORED` pueden usarse en índices

# Columnas calculadas: ejemplo

---

Generar los *tags* separados por comas:

```
ALTER TABLE Posts ADD COLUMN tags_csv TEXT GENERATED ALWAYS AS  
  (REPLACE(REPLACE(REPLACE(Tags, '><', ', '), '<', ''), '>', '')) VIRTUAL;
```

O, como se verá más tarde, como un *array* JSON:

```
ALTER TABLE Posts ADD COLUMN tags_array JSON GENERATED ALWAYS AS  
  (CAST(CONCAT('[',  
               REPLACE(REPLACE(REPLACE(Tags, '><', '"', '"'), '<', '"'), '>', '"'),  
               ']' )  
        AS JSON)  
  ) STORED;
```

# Ejemplo Stack Overflow

---

```
CREATE TABLE Posts (  
  Id INT,  
  PostTypeId INT,  
  AcceptedAnswerId INT,  
  ParentId INT,  
  ...  
  OwnerUserId TEXT,  
  ...  
  PRIMARY KEY(Id),  
  FOREIGN KEY (ParentId) REFERENCES Posts(Id),  
  FOREIGN KEY (OwnerUserId) REFERENCES Users(Id)  
);
```

# Ejemplo Stack Overflow (ii)

---

```
CREATE TABLE Comments (  
    Id INT,  
    PostId TEXT,  
    ...  
    UserId TEXT,  
    PRIMARY KEY(Id),  
    FOREIGN KEY (PostId) REFERENCES Posts(Id),  
    FOREIGN KEY (UserId) REFERENCES Users(Id)  
);
```

# Vistas

---

- Las vistas son consultas parciales que se pueden reutilizar después
- Representan patrones que se pueden sustituir en la consulta donde se usan
- Es decir, donde se usa la vista se podría sustituir por su definición
- Se crean con `CREATE OR REPLACE VIEW v AS SELECT ...`
- Dependiendo del SGBDR se generan vistas persistentes (guardadas en disco) o en memoria

# Vistas: ejemplo

---

Sólo preguntas (o sólo respuestas):

```
CREATE OR REPLACE VIEW Questions AS SELECT * FROM Posts WHERE PostTypeId = 1;
```

Después se puede utilizar como si fuera una tabla:

```
SELECT * FROM Questions WHERE Title LIKE '%Java%';
```

Las vistas se pueden utilizar para eliminar y para insertar valores, siempre que no estén formadas por más de una tabla con `JOIN`.

# Expresiones de tabla comunes (CTE)

Una CTE (*Common Table Expression*) da nombre al resultado de una consulta durante una sola sentencia:

```
WITH PreguntasPorAutor AS (  
    SELECT OwnerUserId, COUNT(*) AS NumPreguntas  
    FROM Posts  
    WHERE PostTypeId = 1  
    GROUP BY OwnerUserId  
)  
SELECT * FROM PreguntasPorAutor WHERE NumPreguntas >= 10;
```

- Permite dividir una consulta compleja en pasos legibles
- Se pueden definir varias CTE separadas por comas y hacer que unas usen otras
- No crea una tabla ni una vista persistente
- El nombre no implica que el motor materialice el resultado: lo decide el optimizador

# Varias CTE en una consulta

```
WITH Preguntas AS (  
    SELECT Id, OwnerUserId FROM Posts WHERE PostTypeId = 1  
)  
, Respuestas AS (  
    SELECT ParentId, COUNT(*) AS NumRespuestas  
    FROM Posts WHERE PostTypeId = 2  
    GROUP BY ParentId  
)  
SELECT p.Id, p.OwnerUserId, COALESCE(r.NumRespuestas, 0)  
FROM Preguntas AS p  
LEFT JOIN Respuestas AS r ON r.ParentId = p.Id;
```

Cada CTE debe tener una granularidad clara. Aquí `Respuestas` produce como máximo una fila por pregunta antes de realizar la unión.

# Data Manipulation Language

Modificación de estado

# INSERT

---

- Permite insertar filas en las tablas
- Puede opcionalmente especificar las columnas sobre las que actúa
- También, opcionalmente, pueden obtenerse los valores de una consulta

```
INSERT INTO tabla[(col1,col2, ...)] VALUES (val1, val2, ...), ...;
```

```
INSERT INTO tabla[(col1,col2, ...)] SELECT ...;
```

# UPDATE

---

- Actualiza filas de una tabla
- La cláusula `SET` especifica qué columnas se modifican y los nuevos valores
- La cláusula `WHERE` selecciona las filas

```
UPDATE tabla SET att1 = valor1, att2 = valor2, ...  
WHERE ...;
```

# UPDATE: valores antiguos y JOIN

---

Se puede utilizar el valor antiguo de los campos en la cláusula SET :

```
UPDATE tabla SET att1 = att1 + 1, att2 = ...  
WHERE ...;
```

También se puede utilizar otra tabla como fuente, con un INNER JOIN :

```
UPDATE tabla  
JOIN otra ON tabla.val = otra.otroval  
SET tabla.att1 = otra.otroatt1, ...  
WHERE ...;
```

# DELETE

---

- Elimina filas de una tabla
- Similar a `SELECT`, con una sola tabla en `FROM`
- La cláusula `WHERE` selecciona las filas

```
DELETE FROM tabla WHERE condición;
```

# Gestión de JSON en SQL

# Introducción

---

- Los SGBDR actuales permiten la integración con JSON, lo cual compararemos con las bases de datos documentales que veremos después
- Las tablas pueden tener columnas de tipo JSON
- Los documentos JSON pueden tener estructuras complejas
- Se pueden hacer consultas sobre los documentos JSON y sus campos
- Se pueden crear índices para acelerar las búsquedas usando campos de tipo JSON
- La sintaxis es ligeramente diferente en distintos SGBDR, aunque el estándar SQL la incluye desde los 2020

# Esquema mínimo (MySQL)

---

Columna tipo JSON y columnas generadas:

```
CREATE TABLE Posts (  
  Id BIGINT PRIMARY KEY,  
  CreationDate TIMESTAMP,  
  Score INT,  
  doc JSON -- documento completo  
);  
CREATE TABLE Users (  
  Id BIGINT PRIMARY KEY,  
  doc JSON  
);  
-- Columna generada para indexar OwnerUserId  
ALTER TABLE Posts ADD COLUMN OwnerUserId BIGINT  
  GENERATED ALWAYS AS (JSON_EXTRACT(doc, '$.OwnerUserId')) STORED;  
CREATE INDEX idx_posts_owner ON Posts(OwnerUserId);
```

# Inserción JSON

---

MySQL:

```
INSERT INTO Posts (Id, CreationDate, Score, doc) VALUES (  
    1, NOW(), 10,  
    JSON_OBJECT('Title', 'How to X?', 'Body', '...', 'OwnerUserId', 42, 'Tags', JSON_ARRAY('sql', 'json'))  
);
```

Equivalente en SQLite (campo `TEXT`):

```
INSERT INTO Posts (Id, CreationDate, Score, doc) VALUES (  
    1, datetime('now'), 10,  
    '{"Title":"How to X?","Body":"...","OwnerUserId":42,"Tags":["sql","json"]}'  
);
```

# Operadores -> y ->> (MySQL)

---

- `doc -> 'key'` devuelve JSON (tipo `JSON` )
- `doc ->> 'key'` devuelve texto (tipo `TEXT` )

```
-- JSON_EXTRACT devuelve JSON; JSON_UNQUOTE(JSON_EXTRACT) devuelve texto
SELECT JSON_EXTRACT(doc, '$.Title') FROM Posts WHERE id = 1;
SELECT JSON_UNQUOTE(JSON_EXTRACT(doc, '$.Title')) AS Title FROM Posts WHERE id = 1;
-- En MySQL se pueden usar los operadores (->, ->>)
SELECT doc->'$.Title', doc->>'$.Title' FROM Posts WHERE id = 1;
>>> "How to X?",How to X?
```

# Operadores `->` y `->>` (SQLite)

---

```
SELECT JSON_EXTRACT(doc, '$.Title') AS title_text FROM posts WHERE id = 1;

-- Propiedad anidada (si doc contiene {"owner": {"id": 42}})
SELECT doc -> '$.owner' ->> '$.id' FROM Posts WHERE id = 1;

-- O también
SELECT JSON_EXTRACT(doc, '$.owner.id') FROM Posts WHERE id = 1;

-- O incluso
SELECT JSON_EXTRACT(JSON_EXTRACT(doc, '$.owner'), '$.id') FROM Posts WHERE id = 1;
```

## JSON\_TABLE *expande arrays*

---

```
-- Expandir tags con JSON_TABLE
SELECT p.id, jt.tag
FROM Posts p,
JSON_TABLE(p.doc, '$.tags[*]' COLUMNS (tag TEXT PATH '$')) AS jt
WHERE p.id = 1;
```

# Equivalentes en SQLite

---

```
-- Propiedad anidada
SELECT json_extract(doc, '$.owner.id') FROM posts WHERE id = 1;

-- Expandir tags
SELECT p.id, t.value AS tag
FROM posts p, json_each(p.doc, '$.tags') AS t
WHERE p.id = 1;
```

# Búsquedas: operador de contención (MySQL)

---

```
-- Buscar ownerId
SELECT * FROM posts WHERE JSON_UNQUOTE(JSON_EXTRACT(doc, '$.ownerUserId')) = '42';

-- Buscar tag dentro de array (JSON_CONTAINS)
SELECT * FROM posts WHERE JSON_CONTAINS(doc, '"sql"', '$.tags');
```

# Búsquedas: operador de contención (SQLite)

---

```
-- Buscar ownerId
SELECT * FROM posts WHERE json_extract(doc, '$.ownerUserId') = 42;

-- Buscar tag: usar json_each para comprobar existencia
SELECT p.*
FROM posts p
WHERE EXISTS (
    SELECT 1 FROM json_each(p.doc, '$.tags') AS t WHERE t.value = 'sql'
);
```

# JSON\_TABLE (MySQL)

---

MySQL tiene `JSON_TABLE`, que mapea JSON a columnas:

```
SELECT *  
FROM posts,  
JSON_TABLE(doc, '$' COLUMNS (  
  Title VARCHAR(200) PATH '$.Title',  
  Body TEXT PATH '$.Body',  
  ownerUserId BIGINT PATH '$.ownerUserId'  
)) AS jt;
```

# SQLite: JSON1

SQLite almacena JSON como texto y provee la extensión JSON1, con funciones como `json_extract`, `json_each`, `json_tree`, `json_array_length`, etc.

```
-- Expandir array tags con json_each
SELECT p.id, t.value AS tag
FROM posts p, json_each(p.doc, '$.tags') AS t
WHERE p.id = 1;

-- Extraer comentarios (json_each sobre array de objetos)
SELECT p.id,
       json_extract(c.value, '$.id') AS comment_id,
       json_extract(c.value, '$.body') AS comment_body
FROM posts p, json_each(p.doc, '$.comments') AS c
WHERE p.id = 1;
```

# Ejemplo: Posts con usuario (MySQL)

```
INSERT INTO Posts (Id, CreationDate, PostTypeId, OwnerUserId, doc)
SELECT
    p.Id, p.CreationDate, p.PostTypeId, p.OwnerUserId,
    JSON_OBJECT(
        'post', JSON_OBJECT(
            'Title', p.Title, 'Body', p.Body, 'Score', p.Score, ...
        ),
        'user', CASE
            WHEN u.Id IS NOT NULL THEN JSON_OBJECT(
                'Id', u.Id, 'DisplayName', u.DisplayName,
                'Reputation', u.Reputation, ...
            )
            ELSE JSON_OBJECT('Id', NULL, 'DisplayName', 'Usuario eliminado')
        END
    ) as doc
FROM Posts p
LEFT JOIN Users u ON p.OwnerUserId = u.Id;
```

# Consejos de modelado

---

- Si un campo va a usarse frecuentemente para filtrado u orden: mantenerlo como columna separada además del JSON
- JSON es ideal para documentos semiestructurados y variantes entre registros
- Normaliza cuando hay relaciones 1:N que se consultarán frecuentemente

# Cuestiones de modelado y eficiencia

Modelado conceptual

# Modelado conceptual

---

- Al modelar transformamos las entidades del «mundo» en entidades del dominio de modelado
- Existen diferentes formas de modelar; las más generalizadas:
  - Modelado basado en objetos
  - Modelado basado en entidades y relaciones (ER)
  - Modelado relacional
- Cada forma de modelado posee:
  - Unos **conceptos** (estructuras)
  - Unas **restricciones**
  - Unas **operaciones** para el acceso a los datos

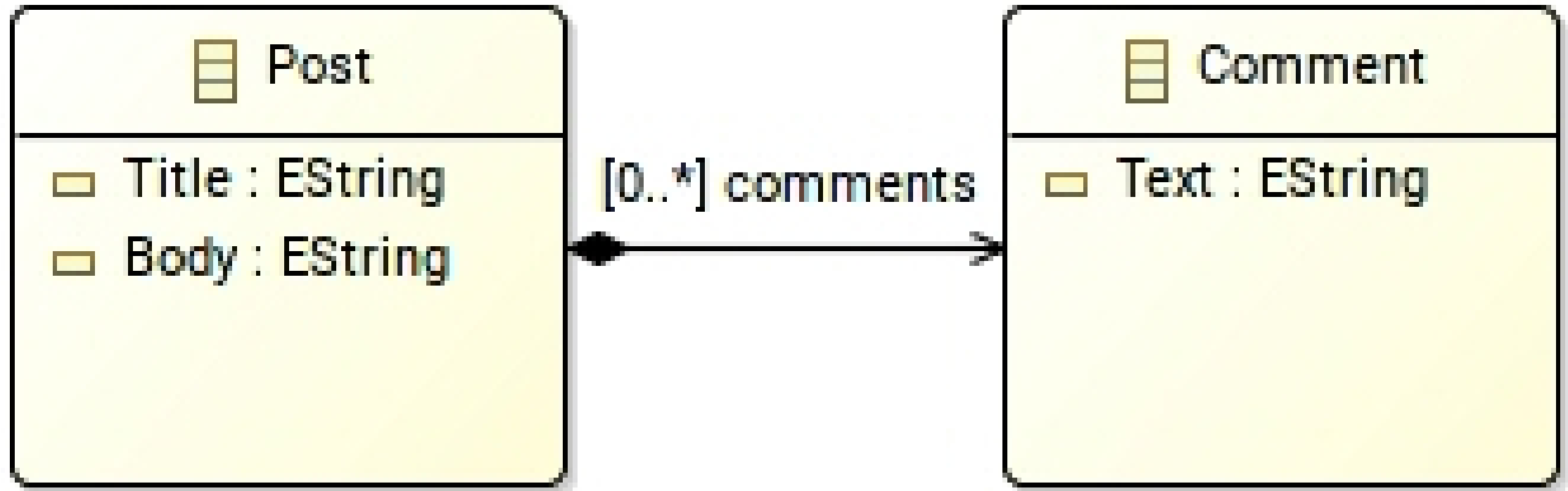
# Modelado conceptual: conceptos base

---

En general, la mayoría de las técnicas de modelado de datos se basan en:

- **Entidades:** elementos del dominio a modelar con entidad e identidad propia
- **Relaciones:** relaciones entre elementos del dominio

# Modelado basado en objetos



En este modelo, el dominio se representa con **objetos** y **relaciones** de herencia, clientela y composición (contenido).

# Modelado basado en objetos: clases

---

Correspondería a un código de definición de clases parecido a:

```
class Post:
    def __init__(self):
        self.Title = ""
        self.Body = ""
        self.Comments = []

class Comment:
    def __init__(self):
        self.Text = ""
```

# Uso del modelo en un lenguaje OO

---

```
# Base de datos de Posts
stackoverflow = []

# A través de la página web, nuevo Post...
new_post = new Post();
new_post.Title = request.Title
new_post.Body = request.Body
stackoverflow.append(new_post)

# A través de la página web, nuevo comentario
# sobre un Post que se mostró en la web...
new_comment = new Comment
new_comment.Text = request.Text
request.Post.Comments.append(new_comment)
```

# Navegación

---

```
# Dado un Post, mostrar el texto
# de todos los comentarios
for comment in post.Comments:
    print comment.Text

# Número medio de comentarios por Post
nc = 0
for post in stackoverflow:
    nc = nc + len(post.Comments)
print float(nc) / len(stackoverflow)

# Número medio de comentarios por Post (alt)
print sum(map(lambda p : len(p.Comments), stackoverflow))
      / float(len(stackoverflow))
```

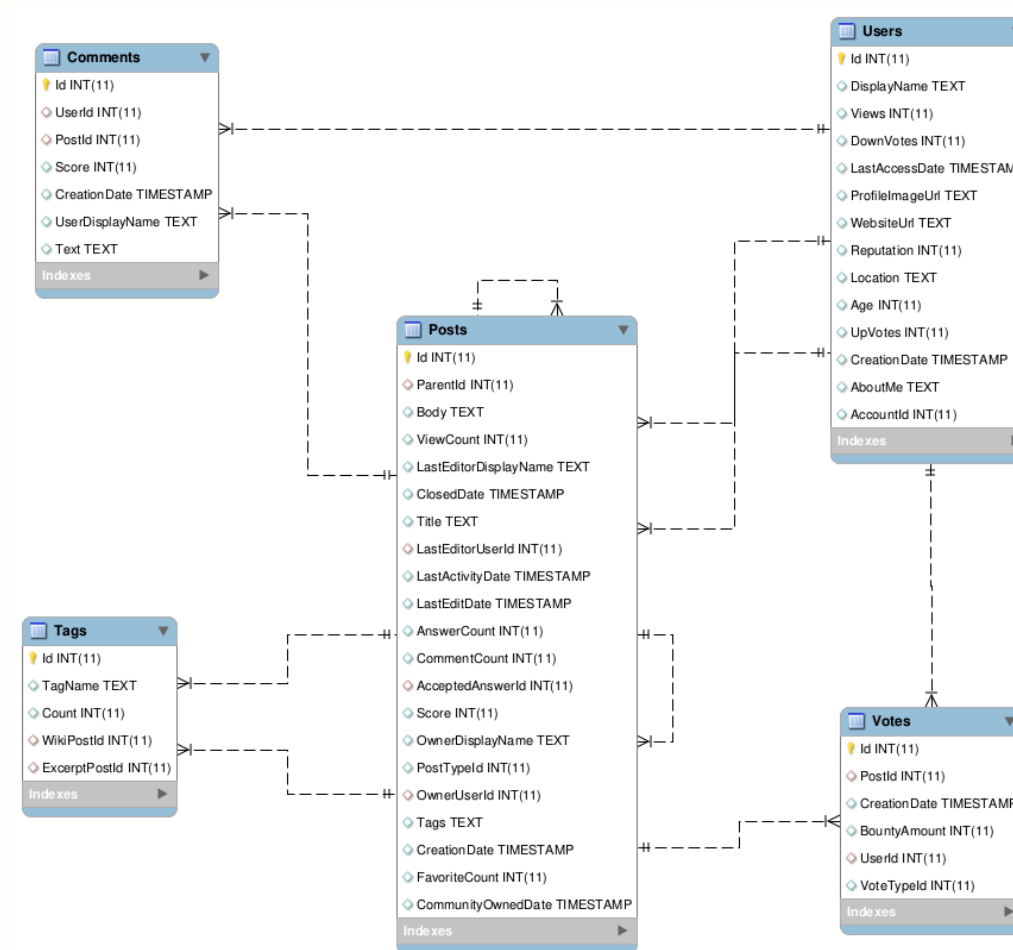
# Modelado basado en objetos: valoración

---

- El modelado es más natural (se modelan objetos que pueden corresponder con el mundo real)
- Se incluyen relaciones de herencia, parte-de, etc.
- Se permiten relaciones de contenido, jerárquicas
- Se permiten colecciones de objetos
- El tiempo de vida (*lifespan*) de una entidad se liga a las relaciones de contenido:
  - Entidades dependientes de un «padre»: contenido
  - Entidades con vida independiente: referencia (clientela)

# Modelado relacional

# Modelado relacional: Stack Overflow



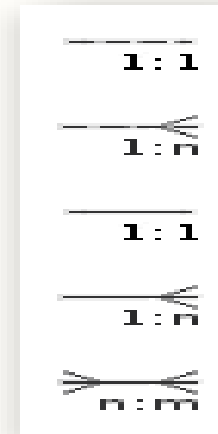
# Modelado relacional

---

- Caso particular de modelado conceptual que utiliza las abstracciones del **modelo relacional**:
  - Tablas para describir entidades del dominio
  - Relaciones entre entidades (tablas)
  - Restricciones (claves ajenas, tipos básicos)
- El modelado también se hace en base a **entidades y relaciones**:
  - Siguiendo las restricciones vistas en la sesión anterior
  - Aplicando los conceptos del modelado de datos conceptual

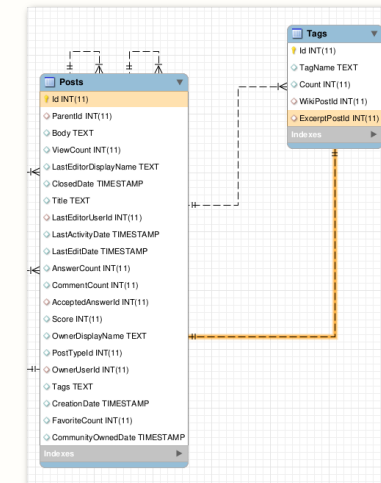
# Relaciones entre entidades

- **Cardinalidad y dependencia**
- Cardinalidad: las relaciones pueden ser **1:1**, **1:n** o **n:m**, e indican el grado de relación de una entidad con el resto
- Dependencia (tiempo de vida):
  - **Identifying** (dependientes): el tiempo de vida de la entidad enlazada depende de la entidad padre
  - **Non-identifying** (no dependientes): las entidades tienen vida propia



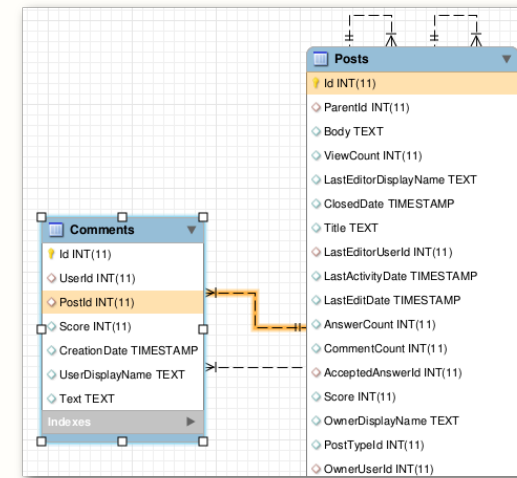
# Relaciones 1:1

- **Conceptualmente** existe un Post que hace de *excerpt* (resumen, descripción) de un Tag
- Posts y Tags, con respecto a «Excerpt», es **1:1**
- En el modelado relacional equivale a una clave ajena que puede estar en una de las dos tablas o en ambas



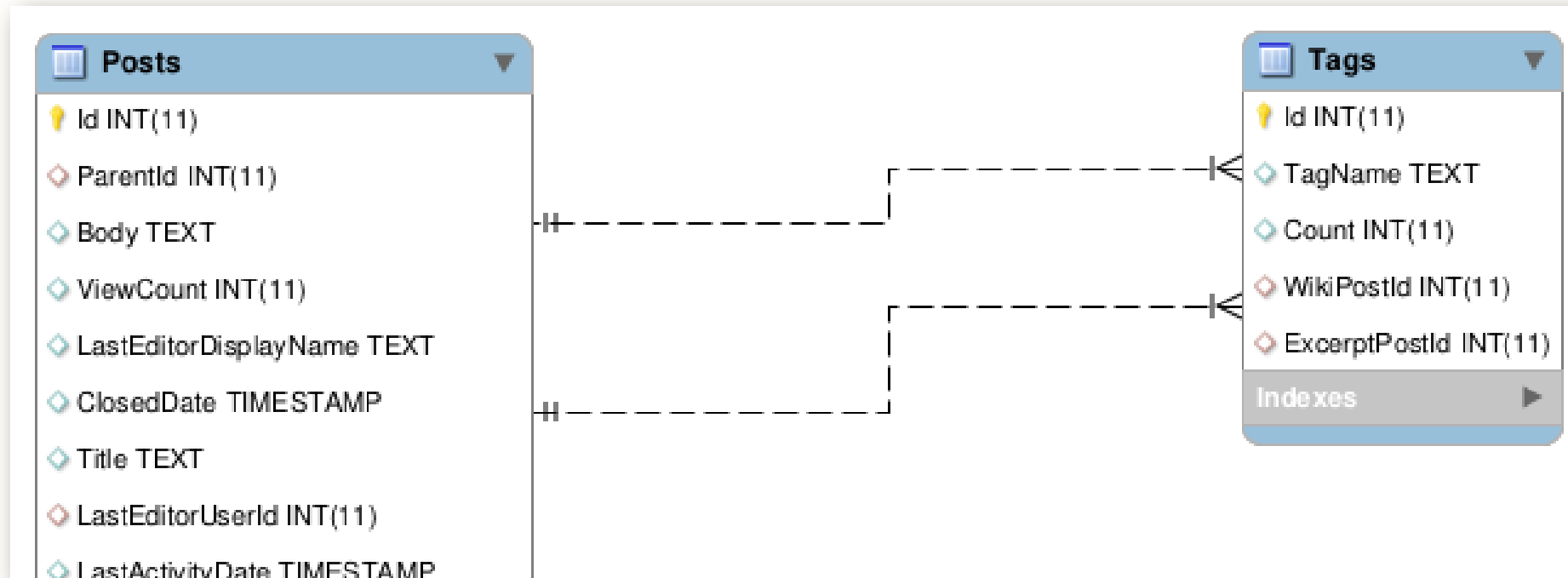
# Relaciones 1:n (uno a muchos)

- Conceptualmente se pueden hacer **comentarios** sobre un **Post**
- Posts y Comments tienen una relación **1:n**
- Puede ser dependiente o no (se ve después)
- En el modelado relacional equivale a una clave ajena en la clase dependiente, o bien a una tabla separada (más adelante)



# Relaciones n:m (muchos a muchos)

En Stack Overflow esta relación está desnormalizada (más adelante). Sin embargo, podemos construir la relación que existe entre **Posts** y **Tags**: sería una relación **n:m**.



# Relaciones n:m (ii)

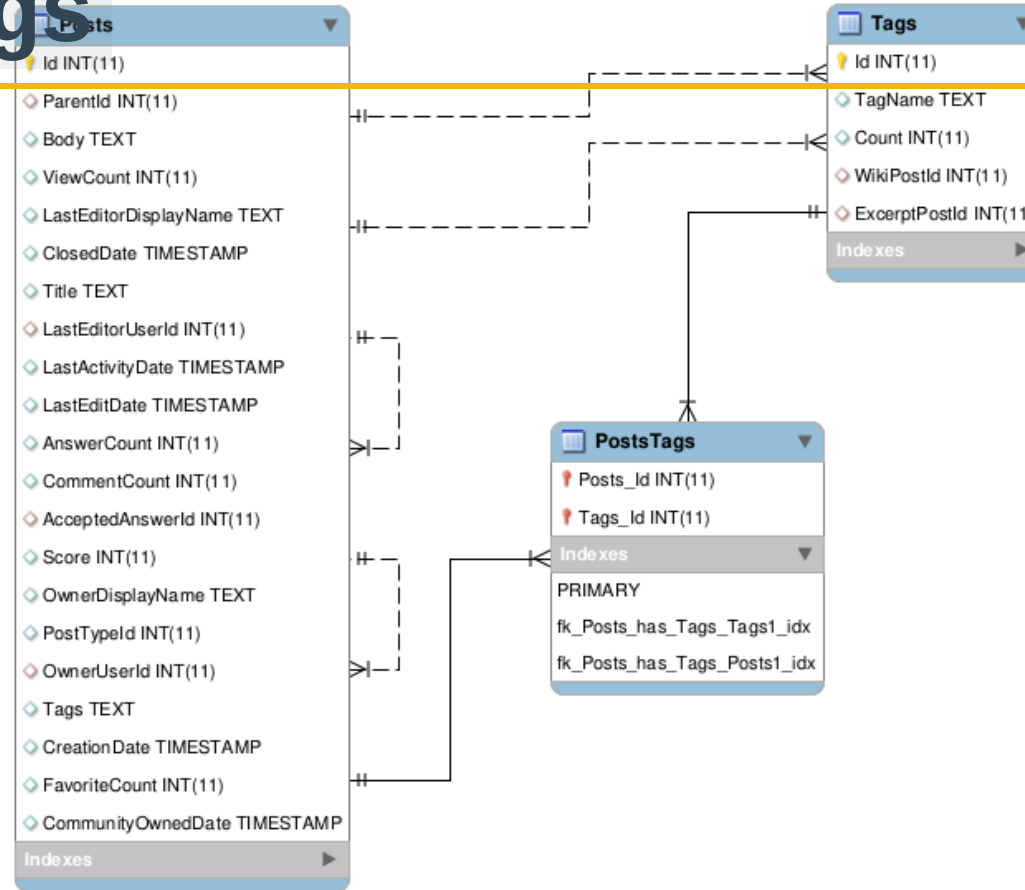
---

- Conceptualmente, es una relación ***n:m***
- Un Post puede tener muchos Tags
- Y un Tag puede aparecer en muchos Posts
- ¿Cómo se obtendría?

# Relaciones n



## tabla PostTags



# Relaciones n:m (iii)

---

- **Ahora bien:**
  - El modelo relacional **no permite añadir esa relación tal cual**
  - Requiere del uso de una **entidad artificial** (*gap* semántico)
- ¿Con qué se corresponde la tabla `PostTags` en el mundo real?
- Ahora la relación es **identifying** 😊
- El caso anterior **1:n** es un caso especial de éste, por lo que también se podría implementar igual

# Relaciones dependientes vs. no dependientes

---

- Son más comunes las **no dependientes**
- Generalmente las relaciones **dependientes** se dan en casos de relaciones **1:n** (p. ej. Posts/Comments)
  - La entidad «1» se considera la entidad «**padre**» (**Posts**)
  - Y la otra, la entidad dependiente (**Comments**)
- La entidad dependiente está supeditada a la entidad padre

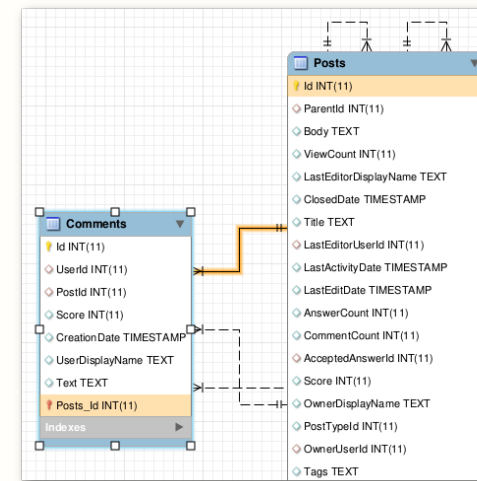
# Relaciones dependientes (ii)

---

- Su tiempo de vida también está supeditado:
  - Cuando la entidad padre se elimina, la entidad dependiente no tiene sentido por sí misma
  - Generalmente indica una restricción de clave ajena con cascada en *delete* y *update*
- Estrategias de implementación relacional:
  - Una tabla aparte (el caso **1:n** es un caso particular de **n:m**)
  - Una clave ajena en la entidad **n** (dependiente o no) que referencia a la entidad **1**
- En las relaciones **dependientes**, dos alternativas:
  - i.** Clave primaria independiente para la entidad dependiente + clave ajena
  - ii.** Clave primaria que incluye la clave dependiente y la clave ajena

# Relaciones dependientes (1:n)

- La clave primaria de **Comments** **incluye** a la clave ajena de **Posts**
- La clave primaria es (Id, Posts\_Id)
- La identificación de un **Comment** sólo tiene sentido **dentro** de un Post
- Al eliminar un Post, el comentario no tiene sentido
- P. ej.: Post 1000, Comentario 1: (clave <1,1000>)



**Eficiencia**

# Data Access Hitting a Wall



## Current practice based on data download (FTP/GREP)

### Will not scale to the datasets of tomorrow

- You can GREP 1 MB in a second
- You can GREP 1 GB in a minute
- You can GREP 1 TB in 2 days
- You can GREP 1 PB in 3 years.
- Oh!, and 1PB ~5,000 disks
- At some point you need **indices** to limit search
- **parallel** data search and analysis
- This is where databases can help
- You can FTP 1 MB in 1 sec
- You can FTP 1 GB / min (~1\$)
- ... 2 days and 1K\$
- ... 3 years and 1M\$



[slide src: Jim Gray]

# Agregar antes de unir relaciones 1:n

---

Una publicación puede tener varios comentarios y votos. Unir ambas relaciones directamente multiplica sus filas:

```
1 publicación × 3 comentarios × 4 votos = 12 filas
```

Aplicar `COUNT()` o `SUM()` después contaría varias veces los mismos hechos. La solución es agregar cada relación por separado hasta obtener **una fila por publicación** y unir después esos resultados. Si la granularidad buscada fuese una fila por pregunta incluyendo sus respuestas, cada agregado tendría que reducirse primero a `PreguntaId`.

# Agregar antes de unir relaciones 1:n (consulta)

---

```
WITH C AS (  
    SELECT PostId, COUNT(*) AS NumComentarios  
    FROM Comments GROUP BY PostId  
)  
, V AS (  
    SELECT PostId, COUNT(*) AS NumVotos  
    FROM Votes GROUP BY PostId  
)  
SELECT p.Id, COALESCE(c.NumComentarios, 0), COALESCE(v.NumVotos, 0)  
FROM Posts AS p  
LEFT JOIN C AS c ON c.PostId = p.Id  
LEFT JOIN V AS v ON v.PostId = p.Id;
```

# Índices

---

- Sea con SQL o con datos *raw*, la búsqueda no se puede realizar secuencialmente
- Los **índices** se utilizan para acelerar esta búsqueda
- El concepto de índice está presente en casi todas las bases de datos (también es un eje principal en las NoSQL)
- En SQL, los índices se pueden aplicar a los valores de una o más columnas
- El planificador de consultas usa los índices disponibles; es **labor del usuario** definir los adecuados
- Los índices aceleran las búsquedas, pero también ocupan espacio
- Se utiliza `CREATE INDEX`, `ALTER TABLE` o se incluyen en `CREATE TABLE`

# Índices: EXPLAIN

EXPLAIN muestra si una consulta usa un índice:

```
mysql> explain select * from Posts where Id=5;
+----+-----+-----+-----+-----+-----+-----+...
| id | select_type | table | type | possible_keys | key | Id |...
+----+-----+-----+-----+-----+-----+-----+...
| 1 | SIMPLE | Posts | ref | Id | Id | Id |...
+----+-----+-----+-----+-----+-----+-----+...
```

En general, hay tres tipos de índices: basados en árboles balanceados, basados en tablas *hash* e índices *full-text*.

# Índices de árboles balanceados

---

- Mantienen la clave ordenada dentro de un árbol balanceado (B-Tree, B+, etc.)
- Aceleran búsquedas con los operadores `=`, `<` y `>`, y rangos (`BETWEEN`)
- También `LIKE` que no empiece con comodín
- Puede ayudar en `ORDER BY`

# Índices compuestos y prefijo izquierdo

Un índice B-tree compuesto se ordena lexicográficamente por sus columnas:

```
CREATE INDEX idx_autor_fecha  
ON Posts (OwnerId, CreationDate);
```

Puede localizar eficientemente consultas por:

- OwnerId
- OwnerId y CreationDate

En general, no permite localizar directamente sólo por CreationDate : falta la primera columna del índice. Es la **regla del prefijo izquierdo**.

El orden debe elegirse a partir de los patrones de acceso, no del orden en que las columnas aparecen escritas en WHERE .

# Igualdades antes del primer rango

Con un índice (A, B, C), el caso más favorable encadena igualdades y después un rango:

```
WHERE A = 7 AND B = 3 AND C >= 100
```

El árbol puede acotar el intervalo mediante las tres columnas. En cambio:

```
WHERE A = 7 AND B >= 3 AND C = 100
```

el rango sobre B suele ser el último que acota el recorrido. C todavía puede evaluarse desde el índice —por ejemplo, mediante *index condition pushdown*—, pero normalmente no reduce el intervalo que hay que recorrer.

Por eso suelen colocarse primero las columnas comparadas por igualdad y después la columna de rango, siempre contrastándolo con las consultas reales.

# Predicados sargables

Un predicado es **sargable** cuando el motor puede convertirlo en un intervalo de búsqueda sobre el valor almacenado en el índice:

```
-- Sargable
WHERE CreationDate >= '2016-01-01'
      AND CreationDate < '2017-01-01'

-- Mismo criterio lógico, pero no sargable sobre CreationDate
WHERE YEAR(CreationDate) = 2016
```

Envolver la columna indexada en una función obliga normalmente a calcular la función por fila. También pueden impedir el acceso eficiente conversiones de tipo implícitas, operaciones aritméticas sobre la columna o patrones

```
LIKE '%texto'.
```

La alternativa es reescribir el predicado sobre la columna original o crear, si el patrón lo justifica, una columna generada con su propio índice.

# Índice candidato no significa índice eficiente

---

- Un índice es **candidato** si sus columnas y operadores permiten usarlo para la consulta; aparece normalmente en `possible_keys` de `EXPLAIN`
- El optimizador puede no elegirlo si estima que recuperará gran parte de la tabla, si otro índice es más selectivo o si el acceso aleatorio costaría más que un recorrido secuencial
- Que aparezca en `key` confirma la elección, no que el plan sea bueno
- Un índice de cobertura puede ahorrar el acceso a las filas de la tabla, pero ocupa más espacio y encarece las escrituras
- `EXPLAIN ANALYZE` ejecuta la consulta y permite contrastar estimaciones, filas recorridas, bucles y tiempo real

La decisión se justifica con el patrón de consulta, la distribución de los datos y el plan medido, no sólo con la existencia del índice.

# Índices de tablas *hash*

---

- Sólo para consultas de igualdad ( = ) y desigualdad ( <=> )
- Son muy rápidos
- No se pueden usar para ORDER BY
- Sólo se pueden usar con valores completos de clave (no permiten prefijos, por ejemplo)

# Índices *full-text*

---

- Usados en búsquedas inversas sobre campos textuales
- Dada una palabra o conjunto de palabras, encuentran la fila o filas en las que el campo indexado contiene esas palabras
- Tienen limitaciones con `LIKE` y comodines ( `%` )
- Por ello los SGBD proveen operadores especiales (p. ej. en MySQL `MATCH( . . . ) AGAINST . . .` , se estudia en prácticas)

# Desnormalización

---

- El modelo relacional basa el modelado de datos en la **normalización**, en sus distintos niveles
- Esto permite aplicar el principio de «sólo almacenar un dato en un sitio», lo que lleva a la eficiencia de datos
- Pero a veces necesitamos también **eficiencia de acceso**, y a veces está reñida con la normalización
- El principal escollo para la velocidad es la unión ( JOIN ) de datos de tablas

# ¿Por qué desnormalizar?

---

- **Mantener la historia:** los datos de los clientes, por ejemplo, pueden cambiar; necesitamos almacenar sus valores, p. ej., al emitir una factura en el pasado. Para eso hay que copiar los datos del cliente en ese momento
- **Mejorar la velocidad de consulta:** a veces, para consultas recurrentes, tenemos que usar uniones de varias tablas. Se puede evitar la unión, por ejemplo, añadiendo claves ajenas a las tablas finales, o bien —como en `Posts.Tags`— replicando la información
- **Precalcular valores necesitados de antemano:** si hay datos que se necesitarán, como medias o totales, podemos calcularlos y almacenarlos de antemano

# Desventajas de desnormalizar

---

- **Espacio en disco:** al igual que con los índices, la copia y duplicación de datos ocupa más espacio
- **Anomalías de datos:** un cambio en un dato puede requerir un cambio en más de una parte de la base de datos para mantener la consistencia. Los SGBDR no están preparados para mantener este tipo de consistencia por sí solos; hay que usar *triggers*, transacciones, etc.
- **Ralentización de las actualizaciones:** dependiendo del *ratio* entre actualizaciones y consultas, puede ser un punto importante
- **Más código, más dependencias, más propenso a errores:** al necesitar más coordinación, hay más dependencias ocultas, más errores y mayor coste de mantenimiento

# Desnormalización: ejemplos en Stack Overflow

---

- En la tabla `Posts` tenemos, por ejemplo, el campo `Tags` , que contiene la lista de los tags. Evita tener que hacer un `JOIN` para obtener sus nombres
- También está el campo `Score` , que aglutina todos los votos positivos recibidos (también está en la tabla `Votes` )

# Particiones

---

- El mecanismo de partición permite partir tablas de forma **lógica**
- Es decir, **transparente al usuario**
- Las tablas partidas tienen ciertas restricciones: por ejemplo, no permiten índices o claves ajenas arbitrarios

# Particionado y claves únicas en MySQL

Todas las columnas usadas por la expresión de particionado deben pertenecer a **cada clave única** de la tabla, incluida la clave primaria.

```
CREATE TABLE PostsPorAnio (  
  Id BIGINT NOT NULL,  
  CreationDate DATETIME NOT NULL,  
  Score INT,  
  PRIMARY KEY (Id, CreationDate)  
)  
PARTITION BY RANGE (YEAR(CreationDate)) (  
  PARTITION p2016 VALUES LESS THAN (2017),  
  PARTITION p2017 VALUES LESS THAN (2018),  
  PARTITION p_futuro VALUES LESS THAN MAXVALUE  
);
```

`PRIMARY KEY (Id)` no sería válida: dos particiones distintas no pueden comprobar por separado una unicidad global sobre `Id`. Esta restricción debe tenerse en cuenta antes de copiar una tabla existente y conservar sus claves.

# Ventajas del particionado

---

- Aceleran las búsquedas y las inserciones (se puede restringir sobre qué particiones actúan)
- Aceleran las búsquedas de rangos
- Permiten almacenar la información histórica de forma eficiente (queda en particiones que se usan rara vez, mientras que los datos actuales usan sólo las últimas particiones)
- Permiten de manera sencilla la **distribución física de los datos en servidores**

# ¿Cuándo usar particionado?

---

- Si casi siempre se hacen consultas sobre la columna de particionado
- Si se tienen datos históricos que se quiere que no «estorben»
- Si los índices son más grandes que la memoria RAM

En MySQL se especifican en `CREATE TABLE` o `ALTER TABLE`:

```
CREATE TABLE <table_name> (<columns>)  
PARTITION BY <type> (<partition_expression>);
```

`<type>` puede ser `RANGE`, `LIST`, `HASH` o `KEY`.

# Particionado **RANGE**

---

```
CREATE TABLE ...  
PARTITION BY RANGE (att) (  
PARTITION p0 VALUES LESS THAN (10000),  
PARTITION p1 VALUES LESS THAN (20000),  
PARTITION p2 VALUES LESS THAN (30000),  
PARTITION p3 VALUES LESS THAN (40000),  
PARTITION p4 VALUES LESS THAN MAXVALUE)
```

# Particionado LIST

---

```
CREATE TABLE ...  
  PARTITION BY LIST (store_id) (  
PARTITION pNorth VALUES IN (2, 8, 12),  
PARTITION pEast VALUES IN (1, 4, 7),  
PARTITION pWest VALUES IN (3, 5, 6, 10),  
PARTITION pSouth VALUES IN (9, 11)  
)
```

# Particionado **HASH** y **KEY**

---

```
CREATE TABLE ...  
PARTITION BY HASH (att)  
PARTITIONS 4;
```

```
CREATE TABLE ...  
PARTITION BY KEY (att)  
PARTITIONS 4;
```

# Consultar particiones

---

```
-- En MySQL 8, la salida normal ya incluye la columna `partitions`.  
EXPLAIN SELECT ...
```

```
SELECT * FROM Tabla PARTITION (p0, p1, ...) WHERE ...
```

# Partition pruning: comprobar, no suponer

El optimizador descarta particiones cuando puede deducir de `WHERE` cuáles podrían contener filas. Para particiones anuales, un rango directo suele expresar bien esa información:

```
EXPLAIN SELECT COUNT(*) FROM PostsPorAnio
WHERE CreationDate >= '2016-01-01'
      AND CreationDate < '2017-01-01';
```

Una transformación arbitraria puede impedir esa deducción:

```
EXPLAIN SELECT COUNT(*) FROM PostsPorAnio
WHERE DATE_FORMAT(CreationDate, '%Y') = '2016';
```

Algunas funciones que coinciden con la expresión de particionado sí son reconocidas por MySQL. Por eso no basta con razonar por intuición: la columna `partitions` de `EXPLAIN` indica las particiones que realmente se leerán.